

Polyglot Big Data Lab

Nine engines. One dataset. One number: 19,252,162.85.

A Docker lab for teaching big data. It runs Hadoop (HDFS), Hive, Spark and Trino, plus PostgreSQL, MySQL, MongoDB, Cassandra, Neo4j, ClickHouse and DuckDB. Every engine is loaded with the same small online-shop dataset, and every engine answers the same revenue question with the same total, so you can compare how each one works.

Quick start

You need **Docker Desktop** (Mac, Windows or Linux). On Windows, turn on WSL 2, run every command in the **Ubuntu (WSL)** terminal, and keep the project inside the WSL file system. Give Docker enough memory (Docker Desktop, Settings, Resources): the default start needs about 3 GB while Spark runs, and every module together needs about 8 GB, so 12 GB is comfortable. Check that Docker works:

```
docker --version
docker compose version
```

1. Get the project

```
git clone https://github.com/choiruzain/polyglot-bigdata-lab.git
cd polyglot-bigdata-lab
```

2. Start the platform

```
sh scripts/platform.sh up
```

The first start builds the images and downloads a few GB. It took about 40 minutes with every module on a fast connection, and later starts take a minute or two. It creates a `.env` file with random passwords for the databases. Never share or commit `.env`. When it finishes, it prints the address of JupyterLab.

3. Open JupyterLab

Open the address it printed, normally `http://127.0.0.1:8888/`. There is no password, because the platform only listens on your own computer. Do not change `BIND_ADDRESS` in `.env` to publish it on a network: without a password, anyone who could reach it could run code on your computer.

4. Load the sample data and run the starter notebook

```
sh scripts/platform.sh load-sample
```

This loads the shop dataset into Hive. In JupyterLab, open `01_hello_spark.ipynb` and press Shift+Enter on its cell. It lists the shop tables and counts the products in each category, using PySpark.

5. Load and try the other engines

The default start runs HDFS, Hive and JupyterLab. To add databases, open `.env`, change the `COMPOSE_PROFILES` line, and run `sh scripts/platform.sh up` again. For example, `COMPOSE_PROFILES=bigdata-lite,sql,r` adds PostgreSQL, MySQL and MongoDB. Load the data for each module you switched on:

Module	Load the shop data	You should see
Hive (default)	<code>sh scripts/platform.sh load-sample</code>	LOADED order_items 59858
sql	<code>sh scripts/platform.sh load-sql</code>	PG_LOADED order_items 59858 and MY_LOADED order_items 59858

Module	Load the shop data	You should see
mongo	sh scripts/platform.sh load-mongo	MONGO_LOADED items 59858
cassandra	sh scripts/platform.sh load-cassandra	CASSANDRA_TOTAL 19252162.85
neo4j	sh scripts/platform.sh load-neo4j	NEO4J_TOTAL 19252162.85
clickhouse	sh scripts/platform.sh load-clickhouse	CLICKHOUSE_TOTAL 19252162.85
DuckDB (no server)	sh scripts/platform.sh demo-duckdb	DUCKDB_TOTAL 19252162.85

Then use the engines together. Each command names the modules it needs, and prints a line ending in the same total, 19252162.85:

```
docker compose exec -T tools spark-submit notebooks/t4_jdbc.py 2>&1 | grep TOTAL_
docker compose exec -T tools spark-submit notebooks/t5_mongo.py 2>&1 | grep TOTAL_
docker compose exec -T tools spark-submit notebooks/t6_neo4j.py 2>&1 | grep TOTAL_
docker compose exec -T trino trino --output-format ALIGNED --file /scripts/federated.sql
```

The first joins Hive, PostgreSQL and MySQL in one Spark query (needs `sql`). The second reads MongoDB (needs `mongo`). The third reads Neo4j (needs `neo4j`). The last is one Trino SQL query across the databases, and prints the revenue per city (needs `trino`, and the databases you switched on).

6. Run Scala

Jupyter here runs Python. Scala runs as a compiled Spark job, using `sbt`, which is already in the notebook container. `notebooks/scala-hello` is a small job that reads the shop data from Hive, so load it first (step 4). The first compile downloads the Scala compiler, so it needs internet and takes a few minutes:

```
docker compose exec tools sh -c 'cd notebooks/scala-hello && sbt -batch package'
docker compose exec tools spark-submit --class HelloScala notebooks/scala-hello/target/scala-2.13/hello
```

Look for the `SCALA_ROWS` line: one row for each product category. Copy the folder to start a project of your own.

7. Stop, restart, erase

```
docker compose stop          # stops everything, keeps your data
sh scripts/platform.sh up    # starts it again
sh scripts/platform.sh reset  # ERASES all data in the platform (asks you to type yes)
```

Notebook only (one container)

If you only want PySpark, the Scala tools and DuckDB with your own folder of notebooks, and no databases, run just the notebook container. It needs about 2 GB. Build the image once, from the project folder:

```
docker build -t polyglot-bigdata-lab images/tools
```

Then run it. Change `<YOUR-LOCAL-FOLDER>` to the folder on your computer that holds your notebooks (for example `$HOME/labs`, or `/mnt/c/Users/YourName/labs` in the Ubuntu WSL terminal):

```
docker run --rm -v <YOUR-LOCAL-FOLDER>:/home/student/notebooks -p 127.0.0.1:8888:8888 -p 127.0.0.1:4040
```

Open <http://127.0.0.1:8888/>. While a Spark session runs, its web page is at <http://127.0.0.1:4040/>. Press **Ctrl+C** to stop. Your notebooks stay in your folder, and anything saved outside it is deleted when the container stops. This container has no databases and no Hive data, so the starter notebook and the Scala

example need the full platform. The 127.0.0.1 in the ports keeps it private to your computer, which matters because it has no password.

Port 8888 already in use? Another subject’s container may be holding it, and `docker run` then stops with “port is already allocated”. See what holds it:

```
docker ps --format '{{.Names}} {{.Image}} {{.Ports}}' | grep ':8888->'
```

Stop it if you recognise it and no longer need it (a container that was started with `--rm` is deleted when it stops):

```
docker stop $(docker ps --format '{{.ID}} {{.Ports}}' | grep ':8888->' | cut -d ' ' -f1)
```

Or leave it alone and use another port: change `-p 127.0.0.1:8888:8888` to `-p 127.0.0.1:8889:8888` in the run command, and open `http://127.0.0.1:8889`. If the first command prints nothing, a program on your computer uses the port. On Mac and Linux, `lsof -nP -iTCP:8888 -sTCP:LISTEN` names it.

What is inside

Module (COMPOSE_PROFILES)	What it starts	Memory when idle
<code>bigdata-lite</code> (default)	HDFS, Hive (metastore and HiveServer2), the notebook container	about 1.8 GB
<code>sql</code>	PostgreSQL 18, MySQL 9.7	about 0.5 GB
<code>mongo</code>	MongoDB 8.3	about 0.3 GB
<code>cassandra</code>	Cassandra 5.0	about 1.4 GB
<code>neo4j</code>	Neo4j 2026.07 (Community)	about 1 GB
<code>clickhouse</code>	ClickHouse 26.8	about 0.5 GB
<code>trino</code>	Trino 483: SQL across the databases	about 1.1 GB
<code>tools</code>	the notebook container only (Spark, Scala, DuckDB)	about 0.1 GB

Everything at once uses about 6 GB idle, and a Spark session adds about 1.25 GB. Measured with Docker Desktop on Apple Silicon. The notebook container has Spark 4.1 (Scala 2.13), Python, sbt, DuckDB and command-line clients for the databases. HDFS is Hadoop 3.5.0 and Hive is 4.2.1.

The shared dataset

A small online shop, generated with a fixed random seed so everyone gets identical data: 1,000 customers, 100 products, 20,000 orders and 59,858 order lines. The question every engine answers: *what is the revenue from delivered orders?* The answer is **19252162.85**, and the loaders print it so you can check.

If something goes wrong

- **The browser says “refused to connect”:** nothing is running. Run `sh scripts/platform.sh up` and open the address it prints. If port 8888 was busy, the start moves JupyterLab to the next free port and the address shows the new one.
- **“port is already allocated”:** `sh scripts/platform.sh up` moves a busy port to the next free one by itself and says which. If it says a port is not set in `.env`, add a line for it, for example `NAMENODE_UI_PORT=9871`. For the notebook-only container, see the port section above.
- **After restarting your computer or Docker, things fail with “unknown host” or a service is missing:** containers do not restart by themselves. Run `sh scripts/platform.sh up`.
- **A start says a container is not healthy:** slow computers can take a while. The start waits a few more minutes, and if the container is still not healthy it prints that container’s log. Run `sh scripts/platform.sh up` again.

- **A container disappears, or things become very slow:** you are probably out of memory. Switch off modules you do not need, or give Docker more memory.
- **Cassandra and Neo4j are slow to start.** Wait for `sh scripts/platform.sh` up to finish before you load data.
- **You changed a password in `.env` after the first start,** and a database now refuses you: the database kept its original password. Run `sh scripts/platform.sh reset`, then start again.

Tips

Run a one-off Hive query from the command line. Beeline may print harmless logging warnings before the result:

```
docker compose exec -T hiveserver2 beeline -u jdbc:hive2://localhost:10000 -n student -e "select count(
```

For an interactive session, use Spark SQL in JupyterLab instead. The interactive beeline prompt draws its tables staggered on some terminals. To use your own folder of notebooks with the full platform, add `NOTEBOOKS_DIR=/full/path/to/your/labs` to `.env` and run `sh scripts/platform.sh up`.

Repository layout

<code>docker-compose.yml</code>	the whole platform
<code>.env.example</code>	settings template (copied to <code>.env</code> on first start)
<code>images/</code>	Dockerfiles: hadoop, hive, tools (with pinned Python packages)
<code>config/</code>	Hadoop, Hive, Tez, Spark, ClickHouse and Trino settings
<code>scripts/</code>	<code>platform.sh</code> (start, stop, load, reset), the data loaders and the checks
<code>notebooks/</code>	the starter notebook, the Scala example and test scripts
<code>data/</code>	sample data is generated here (not committed)
<code>LICENSE, CITATION.cff</code>	

Checks you can run

Four tools live in `scripts/`. Anyone can run them:

<code>sh scripts/check_env.sh</code>	# every setting and script the platform needs is in the repository
<code>bash scripts/clean_clone_test.sh</code>	# clones the committed repo, starts it, loads every engine, checks t
<code>sh scripts/mem.sh</code>	# total memory used by the running containers
<code>sh scripts/pin_bases.sh</code>	# pin (or refresh) the Java base images by digest

Run the clean-clone test after any change you plan to share. It refuses to run with uncommitted changes or while your own platform is running.

Known limitations

- **Tested on Apple Silicon (ARM64) with Docker Desktop only.** Intel and AMD machines and Windows with WSL 2 have not been tested yet.
- **No YARN.** Hadoop here means HDFS; Spark runs in local mode inside the notebook container, and Hive runs its queries on Tez in local mode.
- **No Scala notebook.** Scala runs as a compiled job (step 6).
- **Spark 4 has no Cassandra connector.** Use `cqlsh`, the Python driver, or Trino to reach Cassandra.
- **Trino has no Neo4j or Hive catalog.** Neo4j is reachable from Spark and from its own tools.
- **Teaching-grade security.** JupyterLab has no password, and there is no Kerberos: Hadoop, Hive and Trino trust the user name you give them. Every published port is bound to `127.0.0.1` and every database password is generated per machine, but do not expose this platform to a network.

License and citation

The scripts, configuration and documentation in this repository are released under the MIT License (see LICENSE). To cite this work, use the “Cite this repository” button on GitHub, which reads CITATION.cff.

Third-party software

This repository contains scripts and configuration only. Hadoop, Hive, Spark, Trino, PostgreSQL, MySQL, MongoDB, Cassandra, Neo4j, ClickHouse and DuckDB are downloaded at build time and keep their own licenses.